

Problem

Submissions

Leaderboard

Discussions

Help

HackerRank

PrepareJavaObject Oriented ProgrammingJava Instanceof keyword

Exit Full Screen View

The Java instanceof operator is used to test if the object or instance is an instanceof the specified type.

In this problem we have given you three classes in the editor:

- Student class
- Rockstar class
- Hacker class

In the main method, we populated an ArrayList with several instances of these classes. count method calculates how many instances of each type is present in the ArrayList. The code prints three integers, the number of instance of Student class, the number of instance of Rockstar class, the number of instance of Hacker class.

But some lines of the code are missing, and you have to fix it by modifying only 3 lines! Don't add, delete or modify any extra line.

To restore the original code in the editor, click on the top left icon in the editor and create a new buffer.

Sample Input

```
5
Student
Student
Rockstar
Student
Hacker
```

Sample Output

```
3 1 1
```

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

```
String ret = Integer.toString(a)+" "+ Integer.toString(b)+" "+ Integer.toString(c);
return ret;
}

public static void main(String []args){
    ArrayList mylist = new ArrayList();
    Scanner sc = new Scanner(System.in);
    int t = sc.nextInt();
    for(int i=0; i<t; i++){
        String s=sc.next();
        if(s.equals("Student"))mylist.add(new Student());
        if(s.equals("Rockstar"))mylist.add(new Rockstar());
        if(s.equals("Hacker"))mylist.add(new Hacker());
    }
    System.out.println(count(mylist));
}
```

Line: 38 Col: 1

Upload Code as File

Test against custom input

Run Code

Submit Code

You have earned 10.00 points!

You are now 17 points away from the 4th star for your java badge.

76%

133/160

Congratulations

You solved this challenge. Would you like to challenge your friends?

Next Challenge

Cookie Settings

Accept All Cookies

By clicking "Accept All Cookies", you agree to the storing of cookies on your device to enhance site navigation, analyze site usage, and assist in our marketing efforts. For more information please view our [Privacy Policy](#) [Privacy Policy](#).

Problem List

Accepted X Editorial Solutions Submissions

All Submissions

Accepted 100 / 100 testcases passed

kakuluru_mahitha submitted at Sep 02, 2025 00:26

Analysis Solution

Unlock the Full LeetCode Experience
Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime

8 ms Beats 59.99%

Memory

46.74 MB Beats 91.11%

7ms

8ms

9ms

10ms

11ms

7ms

8ms

9ms

10ms

11ms

Code | Java

```
1 class ParkingSystem {
2     private int[] slots;
3
4     public ParkingSystem(int big, int medium, int small) {
5         slots = new int[]{big, medium, small};
6     }
7
8     public boolean addCar(int carType) {
9
10
11         if (slots[index] > 0) {
12             slots[index]--;
13             return true;
14         }
15
16         return false;
17     }
18 }
```

View more

More challenges

Code

Java Auto

```
1 class ParkingSystem {
2     private int[] slots;
3
4     public ParkingSystem(int big, int medium, int small) {
5         slots = new int[]{big, medium, small};
6     }
7
8     public boolean addCar(int carType) {
9         int index = carType - 1;
10
11         if (slots[index] > 0) {
12             slots[index]--;
13             return true;
14         }
15
16         return false;
17     }
18 }
```

Ln 18, Col 2

Saved

Testcase Test Result

Accepted Runtime: 0 ms

Case 1

Input

["ParkingSystem", "addCar", "addCar", "addCar", "addCar"]

[[1,1,0],[1],[2],[3],[1]]

Output

[null,true,true,false,false]

Expected

[null,true,true,false,false]

Contribute to testcases

Problem List

Accepted

Editorial

Solutions

Submissions

Accepted 57 / 57 testcases passed
kakuluz_mahitha submitted at Sep 02, 2026 09:30

1683.25

Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime

113 ms Beats 30.76%

Memory

57.88 MB Beats 37.38%

Code

Java

```
1 import java.util.*;
2
3 class UndergroundSystem {
4
5     // Customer ID -> check-in information
6     Map<Integer, String> startStation;
7     Map<Integer, Integer> startTime;
8
9     // Route -> total travel time
10    Map<String, Double> totalTime;
11
12    // Route -> number of trips
13    Map<String, Integer> count;
14
15    public UndergroundSystem() {
16        startStation = new HashMap<>();
17    }
18 }
```

More challenges

Code

Java

```
1 import java.util.*;
2
3 class UndergroundSystem {
4
5     // Customer ID -> check-in information
6     Map<Integer, String> startStation;
7     Map<Integer, Integer> startTime;
8
9     // Route -> total travel time
10    Map<String, Double> totalTime;
11
12    // Route -> number of trips
13    Map<String, Integer> count;
14
15    public UndergroundSystem() {
16        startStation = new HashMap<>();
17    }
18 }
```

Testcase

Test Result

Accepted Runtime: 21 ms

Case 1 Case 2

Input

[{"UndergroundSystem", "checkIn", "checkIn", "checkIn", "checkOut", "checkOut", "checkOut", "getAverageTime", "getAverageTime", "checkIn", "getAverageTime", "checkOut", "getAverageTime"}]

Output

[null, null, null, null, null, null, null, 14.00000, 11.00000, null, 11.00000, null, 12.00000]

Expected

Problem List

Description

Accepted

Editorial

Solutions

Submissions

Accepted

73 / 73 testcases passed

Analysis

Solution

kakuluro_mehitha

submitted at Sep 02, 2025 09:32

7583.25

Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime

48 ms

Beats 86.94%

Memory

54.14 MB

Beats 10.49%



Bar chart showing runtime distribution. The x-axis represents runtime in milliseconds (37ms to 70ms) and the y-axis represents percentage (0% to 30%). The distribution is centered around 40ms, with a peak at 40ms reaching approximately 25%.

Code

Java

```
1 import java.util.*;
2
3 class BrowserHistory {
4
5     private ArrayList<String> history;
6     private int current;
7
8     public BrowserHistory(String homepage) {
9         // ...
10    }
11
12    public void visit(String url) {
13        // ...
14    }
15
16    public String back(int steps) {
17        // ...
18    }
19
20    public String forward(int steps) {
21        // ...
22    }
23 }
```

Testcase

Test Result

Accepted

Runtime: 3 ms

Case 1

Input

["BrowserHistory", "visit", "visit", "visit", "back", "back", "forward", "visit", "forward", "back", "back"]

[["leetcode.com"], ["google.com"], ["facebook.com"], ["youtube.com"], [1], [1], [1], ["linkedin.com"], [2], [2], [7]]

Output

[null, null, null, null, "facebook.com", "google.com", "facebook.com", null, "linkedin.com", "google.com", "leetcode.com"]

Expected

[null, null, null, null, "facebook.com", "google.com", "facebook.com", null, "linkedin.com", "google.com", "leetcode.com"]

Problem List

Accepted

Editorial

Solutions

Submissions

Accepted 29 / 29 testcases passed
kakuluru_mahitha submitted at Sep 02, 2020 09:36

7583.25ms

Unlock the Full LeetCode Experience

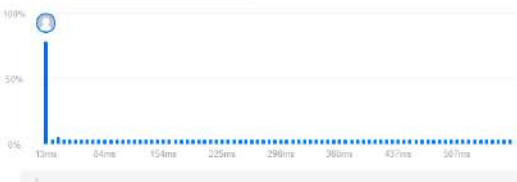
Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime

12 ms | Beat: 97.07%

Memory

53.76 MB | Beat: 51.70%



100%
50%
0%
12ms 64ms 154ms 225ms 296ms 367ms 437ms 507ms

Code

Java

```
1 class MyHashSet {
2
3     private boolean[] set;
4
5     public MyHashSet() {
6         set = new boolean[1000001];
7     }
8
9     public void add(int key) {
10        set[key] = true;
11    }
12
13    public void remove(int key) {
14        set[key] = false;
15    }
16
17    public boolean contains(int key) {
18        return set[key];
19    }
20 }
```

View more

More challenges

Code

Auto

```
1 class MyHashSet {
2
3     private boolean[] set;
4
5     public MyHashSet() {
6         set = new boolean[1000001];
7     }
8
9     public void add(int key) {
10        set[key] = true;
11    }
12
13    public void remove(int key) {
14        set[key] = false;
15    }
16
17    public boolean contains(int key) {
18        return set[key];
19    }
20 }
```

Saved

Ln 20, Col 2

Testcase

Test Result

Accepted Runtime: 0 ms

Case 1

Input

["MyHashSet", "add", "add", "add", "remove", "contains", "add", "add", "add", "remove", "contains", "add", "add", "add", "remove", "contains", "add", "add", "add", "remove", "contains", "add", "add", "add", "remove", "contains"]

[[], [1], [10001], [1], [1], [1], [7], [10007], [7], [7], [7], [123], [10123], [123], [123], [123], [123], [5000], [15000], [5000], [5000], [5000]]

Output

[null, null, null, null, null, false, null, null, null, null, null, false, null, null, null, null, false, null, null, null, null, false]

Expected

[null, null, null, null, null, false, null, null, null, null, null, false, null, null, null, null, false, null, null, null, null, false]

Problem

Submissions

Leaderboard

Discussions

Editor

HackerLand University has the following grading policy:

- Every student receives a *grade* in the inclusive range from 0 to 100.
- Any *grade* less than 40 is a failing grade.

Sam is a professor at the university and likes to round each student's *grade* according to these rules:

- If the difference between the *grade* and the next multiple of 5 is less than 3, round *grade* up to the next multiple of 5.
- If the value of *grade* is less than 38, no rounding occurs as the result will still be a failing grade.

Examples

- grade* = 84 round to 85 (85 - 84 is less than 3)
- grade* = 29 do not round (result is less than 38)
- grade* = 57 do not round (60 - 57 is 3 or higher)

Given the initial value of *grade* for each of Sam's *n* students, write code to automate the rounding process.

Function Description

Complete the function *gradingStudents* with the following parameter(s):

- int grades[n]*: the grades before rounding

Returns

- int[n]*: the grades after rounding

Input Format

The first line contains a single integer, *n*, the number of students.

Each line *i* of the *n* subsequent lines contains a single integer, *grades[i]*.

Constraints

- $1 \leq n \leq 60$
- $0 \leq grades[i] \leq 100$

Sample Input 0

```
4
73
67
38
18
```

```
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
```

```
public static List<Integer> gradingStudents(List<Integer> grades) {
    List<Integer> result = new ArrayList<>();

    for (int grade : grades) {

        // Grades below 38 are not rounded
        if (grade < 38) {
            result.add(grade);
        } else {
            // Find the next multiple of 5
            int nextMultiple = ((grade / 5) + 1) * 5;

            // Round if difference is less than 3
            if (nextMultiple - grade < 3) {
                result.add(nextMultiple);
            } else {
                result.add(grade);
            }
        }
    }
}
```

Line: 75 Col: 1

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

You have earned 10,000 points!

You are now 360 points away from the gold level for your problem solving badge.

4%

490/850

Congratulations

You solved this challenge. Would you like to challenge your friends?

Next Challenge

Using inheritance, one class can acquire the properties of others. Consider the following Animal class:

```
class Animal{
    void walk(){
        System.out.println("I am walking");
    }
}
```

This class has only one method, walk. Next, we want to create a Bird class that also has a fly method. We do this using extends keyword:

```
class Bird extends Animal {
    void fly() {
        System.out.println("I am flying");
    }
}
```

Finally, we can create a Bird object that can both fly and walk.

```
public class Solution{
    public static void main(String[] args){

        Bird bird = new Bird();
        bird.walk();
        bird.fly();
    }
}
```

The above code will print:

```
I am walking
I am flying
```

This means that a Bird object has all the properties that an Animal object has, as well as some additional unique

```
13 class Bird extends Animal{
14     void fly(){
15         System.out.println("I am flying");
16     }
17     void sing(){
18         System.out.println("I am singing");
19     }
20 }
21 }
22
23 public class Solution{
24
25     public static void main(String args[]){
26
27         Bird bird = new Bird();
28         bird.walk();
29         bird.fly();
30         bird.sing();
31     }
32 }
33 }
```

Lines: 12 Cols: 1

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
I am walking
```

1. A class named `Arithmetic` with a method named `add` that takes 2 integers as parameters and returns an integer denoting their sum.
2. A class named `Adder` that inherits from a superclass named `Arithmetic`.

Input Format

Output Format

Sample Output

```
My superclass is: Arithmetic
42 13 20
```

```

1 import java.io.*;
2 import java.util.*;
3 import java.text.*;
4 import java.math.*;
5 import java.util.regex.*;
6
7 class Arithmetic {
8     int add(int a, int b) {
9         return a + b;
10    }
11 }
12
13 class Adder extends Arithmetic {
14 }
15
16 public class Solution{
17     public static void main(String []args){
18         // Create a new Adder object
19         Adder a = new Adder();
20
21         // Print the name of the superclass on a new line
22         System.out.println("My superclass is: " + a.getClass().getSuperclass().getName());
23
24         // Print the result of 3 calls to Adder's 'add(int,int)' method as 3 space-separated integers:
25         System.out.print(a.add(10,32) + " " + a.add(10,3) + " " + a.add(10,10) + "\n");
26     }
27 }
28

```


Problem

Submissions

Leaderboard

Discussions

Bitboard

Generic methods are a very efficient way to handle multiple datatypes using a single method. This problem will test your knowledge on Java Generic methods.

Let's say you have an integer array and a string array. You have to write a **single** method `printArray` that can print all the elements of both arrays. The method should be able to accept both integer arrays or string arrays.

You are given code in the editor. Complete the code so that it prints the following lines:

```
1
2
3
Hello
World
```

Do not use method overloading because your answer will not be accepted.

```
19 public static void main( String args[] ) {
20     Printer myPrinter = new Printer();
21     Integer[] intArray = { 1, 2, 3 };
22     String[] stringArray = { "Hello", "World" };
23     myPrinter.printArray(intArray);
24     myPrinter.printArray(stringArray);
25     int count = 0;
26
27     for (Method method : Printer.class.getDeclaredMethods()) {
28         String name = method.getName();
29
30         if(name.equals("printArray"))
31             count++;
32     }
33
34     if(count > 1)System.out.println("Method overloading is not allowed!");
35
36 }
37 }
```

Line: 4 Col: 1

Upload Code as File

☐ Test against custom input

Run Code

Submit Code



You have earned 15.00 points!
You are now 2 points away from the 4th star for your java badge.



Congratulations

You solved this challenge. Would you like to challenge your friends?



Next Challenge

Problem List

Accepted

Solutions

Submissions

Accepted 49 / 49 testcases passed
kakuluru_mahitha submitted at Sep 02, 2026 09:49

583.25 ms

Unlock the Full LeetCode Experience

Runtime

255 ms Beats 30.90%

Memory

119.76 MB Beats 51.00%



Code

Java

```
1 import java.util.*;
2
3 class ThroneInheritance {
4
5     private String king;
6     private Map<String, List<String>> children;
7     private Set<String> dead;
8
9     // View more
```

More challenges

Code

```
27     return order;
28
29
30     private void dfs(String person, List<String> order) {
31
32         if (!dead.contains(person)) {
33             order.add(person);
34         }
35
36         if (children.containsKey(person)) {
37             for (String child : children.get(person)) {
38                 dfs(child, order);
39             }
40         }
41     }
42 }
```

Testcase

Test Result

Accepted

Runtime: 6 ms

Case 1

Input

```
["ThroneInheritance","birth","birth","birth","birth","birth","birth","getInheritanceOrder","death","ge
tInheritanceOrder"]

[["king"],["king","andy"],["king","bob"],["king","catherine"],["andy","matthew"],["bob","alex"],["bob
","asha"],[null],[null],[null]]
```

Output

```
[null,null,null,null,null,null,null,["king","andy","matthew","bob","alex","asha","catherine"],null,["k
ing","andy","matthew","alex","asha","catherine"]]
```

Expected